

プログラミングI 演習

Computer Programming I: Exercises

第5回 配列

担当

A-P1 辻 愛里

A-P2 北 直樹

A-P3 清水 昭伸

1

本日の講義

本日の目標

- (一次元)配列の利用方法を覚える
- 配列を宣言できる
- #define の利用方法を覚える
- 配列を用いた簡単な演算ができる

本日の資料

1. 前回のおさらい
2. 配列
3. 演習課題
4. 第1回レポート課題

1. 前回のおさらい

for文

•繰り返し処理を記述するための「制御構文」の1つ

1から100000までの数を表示する処理

```
for (x=1; x<=100000; x++)  
{  
    printf("%d¥n", x );  
}
```

補足: 「x++」は 変数「x」を1つ増やす
という演算をします。
(この場合は、x=x+1と同じ結果になる)
「++」をインクリメント演算子と呼びます

初期設定

ループの最初に実行

実行条件式

ループを繰り返す条件

繰り返し処理

ループで繰り返される処理

処理後実行文

繰り返し処理後に実行

3

1. 前回のおさらい

while文

```
x=1;  
while (x <= 100000)  
{  
    printf("%d¥n", x );  
    x++;  
}
```

1から100000までの数を表示する処理

```
for (x=1; x<=100000; x++)  
{  
    printf("%d¥n", x );  
}
```

while文は・・・

実行条件式

ループを繰り返す条件

と

繰り返し処理

ループで繰り返される処理

の2つの部分から構成されている

初期設定

ループの最初に実行

while文の外に移動している

処理後実行文

繰り返し処理後に実行

while文内の繰り返しの一部として最後に記述

4

1. 前回のおさらい

繰り返し回数を適当に決めて実行し、円周率に近づくか確かめてみよう

演習課題4-4 数値計算②: 円周率の計算A

計算方法「グレゴリ・ライプニッツ級数」を用いる

$$4\left\{\frac{1}{1}-\frac{1}{3}+\frac{1}{5}-\frac{1}{7}\cdots\right\}=4\sum_{n=0}^{\infty}\frac{(-1)^n}{2n+1}=\pi$$

基本方針

- ・ Σ の型式になっているので、演習4-3を基本とすればよさそうだ
- ・ $(-1)^n$ はループ1回毎に符号が入れ替わっているだけ
(if 文を用いるか、ある変数「a=-1.0」の符号を毎回入れ替える等)
- ・ まずは Σ 以降の部分を実行し、最後に4を掛ける

問題点

- ・ 0~ ∞ までのループでは終わりが来ない → 回数か精度を決める

円周率: 3.14159 26535 89793 23846 26433 83279 50288 5

1. 前回のおさらい

収束判定をする方法もあるが、反復回数が極端に増える可能性がある。今回は反復回数を指定。

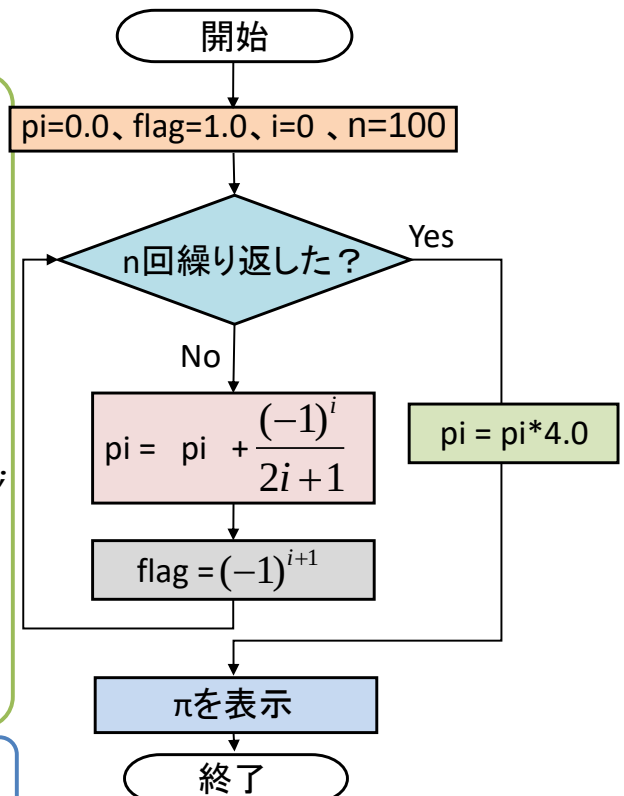
解答例(反復回数を指定)

```
#include <stdio.h>
int main(void){
    int i, n=100;
    double pi, flag;

    /*初期化*/
    pi = 0.0; flag = 1.0;

    /*反復(n回反復)→Σの中身*/
    for(i=0; i<n; i++){
        pi=pi+flag/(2.0*(double)i+1.0);
        flag = -flag; /*符号の反転*/
    }
    pi = pi*4.0; /*4倍に*/
    printf("%d回:π=%1fです。¥n", i, pi);
    return 0;
}
```

型が異なる変数同士の演算が含まれる
→ 原則として精度の高い型に合わせる



1. 前回のおさらい

繰り返し回数を適当に決めて実行し、円周率に近づくか確かめてみよう

演習課題4-5 数値計算③: 円周率の計算B

「マチンの公式」を用いると
少ない反復回数で π を求めることができる

$$16\left\{\frac{1}{5} - \frac{1}{3 \times 5^3} + \frac{1}{5 \times 5^5} - \frac{1}{7 \times 5^7} \cdots\right\} - 4\left\{\frac{1}{239} - \frac{1}{3 \times 239^3} + \frac{1}{5 \times 239^5} - \frac{1}{7 \times 239^7} \cdots\right\} = 16 \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1) \times 5^{(2n+1)}} - 4 \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1) \times 239^{(2n+1)}} = \pi$$

239の〇乗はかなり値が大きい
nが小さくても、きちんと計算できないかも

円周率: 3.14159 26535 89793 23846 26433 83279 50288 7

1. 前回のおさらい

解答例(反復回数を指定)

```
#include <stdio.h>
int main(void) {
    int i, j, n=10; double pi, flag=1.0;
    double A=0.0, B=0.0; /* Aは1つめのΣ計算, Bは2つめのΣ */
    double n_jou;

    for(i=0; i<n; i++){
        for(j=1, n_jou=1; j<=2*i+1; j++) {n_jou = n_jou * 5;}
        A = A + flag/((2.0*(double)i+1.0)*n_jou);
        flag = -flag;
    }
    flag = 1.0;
    for(i=0; i<n; i++){
        for(j=1, n_jou=1; j<=2*i+1; j++) {n_jou = n_jou * 239;}
        B = B + flag/((2.0*(double)i+1.0)*n_jou);
        flag = -flag;
    }

    pi = 16.0 * A - 4.0 * B;
    printf("%d回:π=%lfです。¥n", i, pi);
    return 0;
}
```

$$16 \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1) \times 5^{(2n+1)}} - 4 \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1) \times 239^{(2n+1)}} = \pi$$

実行文が一つの場合は{}は不要だが、
範囲を明確にするために書く

まず、5や239の累乗から求める
→ もっと楽に求める方法はある(後日)

1. 前回のおさらい

発展4-1 精度の測定

グレゴリ・ライプニッツ級数 と マチンの公式 の精度比較

「マチンの公式」を用いると
少ない反復回数で π を求めることができる



これが本当に正しいか否か確かめてみよう

基本方針

- ある繰り返し回数に対して、グレゴリ・ライプニッツ級数を用いた結果と、マチンの公式を用いた結果を出力できるようにする
- 繰り返し回数を1から適当なところまで1ずつ増やし、値の変化を確認する
- (上級者向け)1から入力した繰り返し回数までの結果すべてを出力する
→ 比較しやすい. どちらが先に円周率に近づくかが一目でわかる

9

1. 前回のおさらい

%.10lfとして、小数第10位まで
求めてみた

解答例(測定結果の一部)

繰り返し回数n	グレゴリ・ライプニッツ	マチン
1	4.0000000000	3.1832635983
2	2.6666666667	3.1405970293
3	3.4666666667	3.1416210293
5	3.3396825397	3.1415926824
10	3.0418396189	3.1415926536 (3.14159265359) → %.11lf
100	3.1315929036	—
1,000	3.1405926538	—
10,000	3.1414926536	—
100,000	3.1415826536	—
1,000,000	3.1415916536	—
10,000,000	3.1415925536	—

円周率: 3.14159 26535 89793 23846 26433 83279 50288 10

2. 配列

例題: 試験のデータ検索プログラム

番号	氏名	点数
1	Mayumi	85
2	Ryosuke	74
3	Kohei	70
4	Satoshi	68
5	Takahiro	79
6	Noriaki	62

基本方針

- 6人分のデータ保存する変数の宣言
- 全員分のデータ入力
- 点数を閲覧したい番号を入力
- その人の点数を表示する

11

2. 配列

例題: 試験のデータ検索プログラム

番号	氏名	点数
1	Mayumi	85
2	Ryosuke	74
3	Kohei	70
4	Satoshi	68
5	Takahiro	79
6	Noriaki	62

基本方針

- 6人分のデータ保存する変数の宣言
- 全員分のデータ入力
- 点数を閲覧したい番号を入力
- その人の点数を表示する

```
int main(void)
{
    int Mayumi; int Ryosuke; int Kohei;
    int Satoshi; int Takahiro; int Noriaki;
    int n;
    printf("番号(1)の点数を入力");
    scanf("%d",& Mayumi);

    printf("番号(2)の点数を入力");
    scanf("%d",& Ryosuke);

    ⋮

    printf("番号(6)の点数を入力");
    scanf("%d",& Noriaki);

    printf("何番の点数を見ますか?");
    scanf("%d",&n);

    if(n==1)
        printf("1番の点数は%dです", Mayumi);
    else if(n==2)
        printf("2番の点数は%dです", Ryosuke);
    ⋮
}
```

12

2. 配列

このプログラムの問題点

人数が増えれば増えるほど……

- ・変数宣言を増やさないといけない
- ・入出力を書き足さないといけない



いずれ(すでに)設計として破綻する
同じデータの集合をうまく扱う方法はないか？

```
int main(void)
{
    int Mayumi; int Ryosuke; int Kohei;
    int Satoshi; int Takahiro; int Noriaki;
    int n;
    printf("番号(1)の点数を入力");
    scanf("%d",& Mayumi);

    printf("番号(2)の点数を入力");
    scanf("%d",& Ryosuke);

    ⋮

    printf("番号(6)の点数を入力");
    scanf("%d",& Noriaki);

    printf("何番の点数を見ますか？");
    scanf("%d",&n);

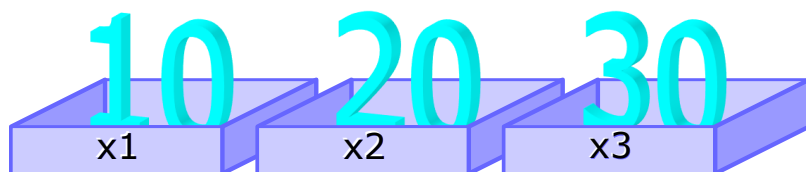
    if(n==1)
        printf("1番の点数は%dです", Mayumi);
    else if(n==2)
        printf("2番の点数は%dです", Ryosuke);
    ⋮
}
```

13

2. 配列

配列: 同じ型の変数をまとめて扱う

複数の変数 (int x1,x2,x3;)



1つの名前に1つの領域が割り当てられる

お互いの関係
「隣の変数」や「次の変数」
等の情報をもっていない

配列: 「int x[3];」



1つの名前に「n個の」領域が割り当てられる
(例の場合は「n=3」)

配列「x」の「i」番目の要素
という考え方ができる

14

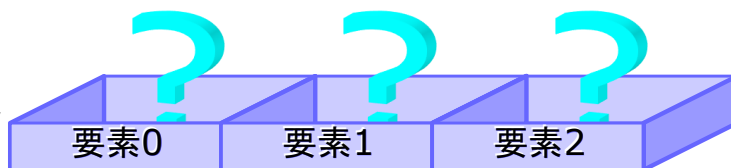
2. 配列

配列の宣言

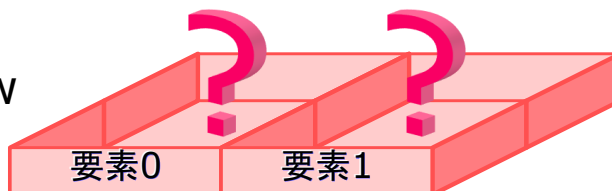
型 配列名[配列の大きさ];

例: `int v[3];`
`double w[2];`

配列v



配列w



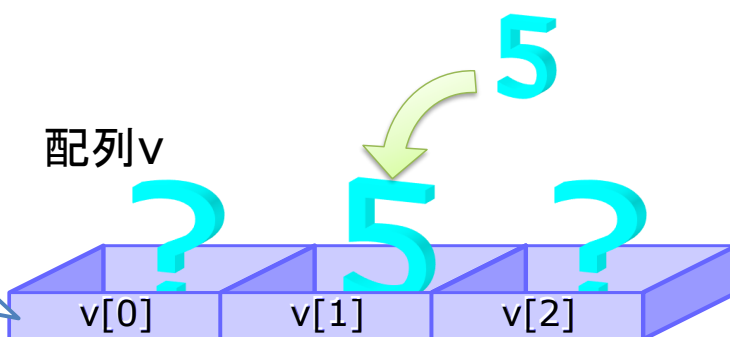
各要素へのアクセス

配列名[要素番号] = 5;

配列「v」の
1番目の要素 → `v[1]=5;`

配列の要素は0から始まる

配列v



15

2. 配列

解答: 試験のデータ検索プログラム (※表の番号を配列の要素番号に合わせて0~5に変更)

番号	氏名	点数
0	Mayumi	85
1	Ryosuke	74
2	Kohei	70
3	Satoshi	68
4	Takahiro	79
5	Noriaki	62

基本方針

- 6人分のデータ保存する変数の宣言
- 全員分のデータ入力
- 点数を閲覧したい番号を入力
- その人の点数を表示する

配列を使うことでプログラムを改善。
人数を増やす場合は、配列の要素数
を増やすだけで良い

```
#include <stdio.h>
int main(void){
    int score[6], n;

    for(n = 0; n < 6; n++)
    {
        printf("番号(%d)の点数を入力", n);
        scanf("%d", &score[n]);
    }

    printf("何番の点数を見ますか?");
    scanf("%d", &n);

    printf("%d番の点数は%dです",
           n, score[n]);
    return 0;
}
```

16

2. 配列(補足1)

配列のサイズ指定は有限の固定値(整数定数式)で行う

```
int n;
```

```
printf("配列のサイズは?");  
scanf("%d",&n);
```

```
int data[n]; エラーは出ないが危険！
```

- メモリの使用量を超える大きすぎる配列は宣言できない

- 配列サイズを変数で指定

→ 任意コードの実行やスタックオーバーフローのリスクがあるので慎重に行う

参考: <https://www.jpcert.or.jp/sc-rules/c-arr32-c.html>
(※可変長配列は「動的メモリ確保」で実現可能→発展7-1)

「#define」文との組み合わせ

```
#include <stdio.h>  
#define DATA_SIZE 20  
int main(void){  
    int data[DATA_SIZE], n;  
  
    for(n=0;n<DATA_SIZE;n++)  
        printf("data[%d]=%d¥n",data[n]);  
    return 0;  
}
```

```
#define [シンボル名] [文字列]
```

- ソースコード中の[シンボル名]をコンパイラが[文字列]に置換する
→ここでは要素数として20が入る
- データのサイズ等を意味のある名前で見えるため、サイズ変更の際に書き換える場所が最小限で済むため、便利

17

2. 配列(補足2)

配列全体の代入(コピー)はできない

```
int dataA[100];  
int dataB[100];
```

```
dataA = dataB; エラー！
```

- 配列を別の配列にコピーすることは、たとえサイズが同じであってもできない
(※第7回「ポインタ」でこの理由が分かります)



```
int dataA[100];  
int dataB[100];
```

```
for(n=0; n<100; n++)  
    dataA[n]=dataB[n];
```

- ループを用いて1つ1つコピーする必要がある
(※第7回「ポインタ」で便利な方法を解説します)

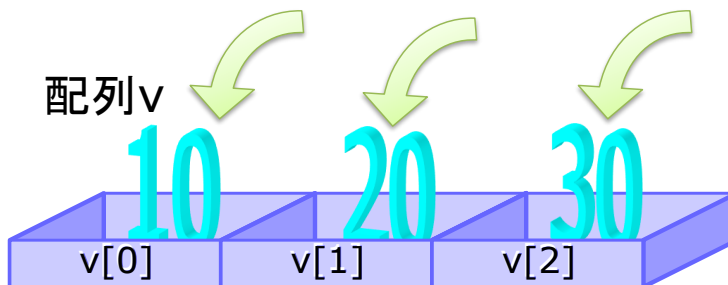
18

2. 配列(補足3)

配列の初期化(※宣言だけだと変数の場合と同じく中身は不定)

```
int v[3]={10,20,30};
```

{~}の中に、「,」で区切って最初に入れておきたい数字を列挙する



補足:まとめて全ての要素をゼロで初期化する方法

```
int v[3] = {};
```

19

ここから演習課題です

3. 演習課題

演習5-1 配列を用いたプログラムの動作確認

```
#include <stdio.h>
int main(void) {
    int score[6], n;

    for(n = 0; n < 6; n++) {
        printf("番号(%d)の点数を入力", n);
        scanf("%d", &score[n]);
    }
    printf("何番の点数を見ますか?");
    scanf("%d", &n);
    printf("%d番の点数は%dです", n, score[n]);

    return 0;
}
```

プログラムを実行して配列の使い方を確認しよう。
定義外の要素番号(例:7)を見ようとする、どうなるかを確認しよう
宣言する配列の要素数も変えてみよう。

21

3. 演習課題

演習5-2 #define の利用

```
#include <stdio.h>
#define YOUSOU 10
int main(void) {
    int score[YOUSOU], n;

    for(n = 0; n < YOUSOU; n++) {
        printf("番号(%d)の点数を入力", n);
        scanf("%d", &score[n]);
    }
    printf("何番の点数を見ますか?");
    scanf("%d", &n);
    printf("%d番の点数は%dです", n, score[n]);

    return 0;
}
```

このプログラムを実行し、配列の要素数が10になることを確認しよう。

22

3. 演習課題

演習5-3 数列に関する課題解決(添字に注意)

$$a_n = 7 \cdot 3^{n-1} - 2 \text{ です}$$

以下の課題を配列を使って解いてみよう

- [1] 数列 $\{a_n\}$ において, $a_1=5$, $a_{n+1}=3a_n+4$ のとき, 第20項(a_{20})は何?
- [2] 数列 $\{a_n\}$ が $a_n=3^n-30$ で表されるとき, 初項(a_1)から第20項(a_{20})までの和は?
- [3] 3で割り切れないすべての正の整数を, 小さいものから並べてできる数列を $a_1, a_2, a_3, \dots, a_n, \dots$ とする. $S_n \geq 600$ となる最小の正の整数 n を求めよ. <2003年 立教大>

$$S_n = (3/2) \cdot (3^n - 1) - 30n \text{ です}$$

累乗の計算を練習しよう

1, 2, 4, 5, 7, 8, ...です. 入試問題なので, 手計算で答えを出せますが, 配列を使ってやってみましょう. 配列の要素数はいくつまで準備する?

23

3. 演習課題

発展5-1 試験の統計処理プログラム

- 10人分の点数を入力する
- 10人分の点数を表示し, 各人の偏差値を表示する

実行例

```
0番の点数:0
1番の点数:10
2番の点数:20
3番の点数:30
4番の点数:40
5番の点数:50
6番の点数:60
7番の点数:70
8番の点数:80
9番の点数:90
0番の点数は0点です。
1番の点数は10点です。
2番の点数は20点です。
3番の点数は30点です。
4番の点数は40点です。
5番の点数は50点です。
6番の点数は60点です。
7番の点数は70点です。
8番の点数は80点です。
9番の点数は90点です。
```

プログラム例(点数の表示だけ)

```
int main(void)
{
    int score[10], i;

    for(i = 0; i < 10; i++) {
        printf("■ ■ ■ ■ ■ ■ ■ ■");
        scanf("%d", &score[i]);
    }

    for(i = 0; i < 10; i++) {
        printf("■ ■ ■ ■ ■ ■ ■ ■");
    }
}
```

24

3. 演習課題

発展5-1 試験の統計処理プログラム

偏差値の計算と表示

全体の平均値

$$\text{偏差値}_i = 10.0 \times \frac{\text{Score}[i] - \overline{\text{Score}}}{\text{標準偏差}} + 50.0 \quad \text{※標準偏差} = \sqrt{\text{分散}}$$

補足: ルート(√)の計算

関数「sqrt」(Square root)を利用する

```
#include <stdio.h>
#include <math.h> /*必要ヘッダファイル*/

int main(void)
{
    double root2 = sqrt(2.0);
    printf("2の平方根は%f\n", root2);
    return 0;
}
```

gccで数学関数を利用するソースコード
(以下では test.c)をコンパイルした際に
未定義エラーがでる場合、

「gcc ./test.c -lm」

のようにコンパイルオプション「-lm」(ハイ
フン・エル・エム)を指定してコンパイルし
てみてください。

偏差値を出すためには
平均値と分散が必要

25

3. 演習課題

発展5-1 試験の統計処理プログラム

平均点の計算

$$\text{平均点}[\overline{\text{Score}}] = \frac{1}{n} \sum_{i=0}^{n-1} \text{Score}[i]$$

(不偏)分散の計算

$$\text{分散}[\sigma^2] = \frac{1}{n-1} \sum_{i=0}^{n-1} (\text{Score}[i] - \overline{\text{Score}})^2$$

補足: $c = (a-b)^2$ の計算

double a,b,c;

c = (a - b)*(a - b);

c = a - b;

c = c*c;

•2つの計算方法がある
•演算記号の数(計算量)
を比べてみよう

プログラム例

```
double average = 0.0;
for(i = 0; i < N; i++)
{
    /*score[n]をaverageに合計*/
}
average = average/N;
printf("平均は%fです\n", average);
```

プログラム例

```
double variance = 0.0;
for(i = 0; i < N; i++)
{
    /*(score[n] - average)^2を合計*/
}
variance = variance/N;
printf("分散は%fです\n", variance);
```

26

3. 演習課題

発展5-1 試験の統計処理プログラム

実行例

0番目の点数:72
1番目の点数:68
2番目の点数:94
3番目の点数:26
4番目の点数:73
5番目の点数:88
6番目の点数:58
7番目の点数:85
8番目の点数:92
9番目の点数:45

試験結果(平均点=70.10点)

0番:72点(偏差値50.87)
1番:68点(偏差値49.04)
2番:94点(偏差値60.90)
3番:26点(偏差値29.90)
4番:73点(偏差値51.32)
5番:88点(偏差値58.16)
6番:58点(偏差値44.48)
7番:85点(偏差値56.79)
8番:92点(偏差値59.98)
9番:45点(偏差値38.56)

27

本日の説明内容はおしまいです。
続いて、第一回目レポートについて説明をします。
説明後に本日の課題に取り組んでください。